



TEL AVIV אוניברסיטת
UNIVERSITY תל אביב

מבני נתונים –

תרגיל בית מעשי 2

מגישים (שם, ת"ז, שם משתמש):

kobiehazon –	יעקב חזון –
itzchakharel -	יצחק הראל –

תאריך הגשה: 4.2.2019

מסמך התייעוד הנלווה לתרגיל מעשי 2 – מימוש ערימת פיבונצ'י

תיעוד

כפי שהתבקשנו, מימשנו את הדרוש באמצעות שתי מחלקות ששמן:

- **HeapNode** שמתארת צומת בודד בערימה.
- **FibonacciHeap** שמתארת את ערימת הפיבונצ'י הכוללת.

כמו כן, כפי שהופיע בקובץ השלד, המחלקה **HeapNode** נכתבה בתור מחלקה פנימית של המחלקה **FibonacciHeap**.

הסבר על מחלקת HeapNode:

כאמור, מחלקה זו מתארת את צמתי הערימה ומכאן שכוללת את השדות הבאים:

- **key** - מטיפוס *int*. תפקידו לשמור את מפתח הצומת.
- **degree** - מטיפוס *int*. תפקידו לשמור את מספר הבנים של הצומת.
- **child** - מטיפוס *FibonacciHeap*. תפקידו להצביע לערימה שמורכבת מילדי הצומת.
- **next** - מטיפוס *HeapNode*. תפקידו לשמור את הצומת הצמודה לצומת זו.
- **mark** - מטיפוס *Boolean*. תפקידו לציין אם הצומת מסומנת כלומר האם אי פעם במהלך התוכנית מחקנו בן שלה.
- **parent** - מטיפוס *HeapNode*. תפקידו לשמור את האבא של אותה צומת כלומר את הצומת שהצומת הנוכחית היא הבן שלה.

יתרה מכך, במחלקה מימשנו את המתודות הבאות:

○ HeapNode(int key) -

מה היא עושה-

בנאי מחלקה זה מקבל טיפוס *int* אשר יהווה את מפתח הצומת ומאתחל את שאר השדות לערכים דיפולטיים. *Child* יאותחל להיות ערימה ריקה.

כיצד היא פועלת-

בונה את האובייקט באופן המתואר לעיל.

סיבוכיות זמן הריצה שלה-

$O(1)$ שכן נעשית רק החזרה של שדה המופע.

○ insertSon(HeapNode other) -

מה היא עושה-

מתודה זו מוסיפה את הצומת שמתקבלת להיות בן חדש של הצומת הנוכחית.

כיצד היא פועלת-

אם child הינו ערימה ריקה אז שדות ה-first וה-last יעודכנו להיות other ואחרת other יכנס להיות צומת ראשונה בערימה של child.

סיבוכיות זמן הריצה שלה-

$O(1)$ שכן הכנסת איבר ראשון בערימה ועדכון השדות לוקחים סיבוכיות זמן ריצה של $O(1)$,

○ getKey() -

מה היא עושה-

מתודה זו מחזירה את המפתח של הצומת הנוכחית.

כיצד היא פועלת-

מחזירה את שדה ה-child של HeapNode.

סיבוכיות זמן הריצה שלה-

$O(1)$ שכן מדובר בהחזרה של שדה בלבד.

הסבר על מחלקת FibonacciHeap:

מחלקה זו מתארת את ערימת הפיבונצ'י ועושה שימוש במופעים של HeapNode וכן במתודות המצויות במחלקה זו.

במחלקה זו מצויים השדות הבאים:

- **size** - מטיפוס *int*. תפקידו הוא לשמור את גודל הערימה (מספר הצמתים הפנימיים שיש בערימה).
- **first** - מטיפוס *HeapNode*. תפקידו להצביע לאיבר הראשון בערימה.
- **last** - מטיפוס *HeapNode*. תפקידו להצביע לצומת המינימלית בעץ האחרון בערימה כלומר לצומת האחרונה ברמה הגבוהה ביותר בערימה.
- **beforeMin** - מטיפוס *HeapNode*. תפקידו להצביע לצומת ששדה ה-*next* שלה מצביע לצומת בעלת המפתח הקטן ביותר בכל הערימה.
- **marked** - מטיפוס *int*. תפקידו הוא לשמור את מספר הצמתים המסומנים בערימה. כלומר מספר הצמתים שערך השדה *mark* שלהם שווה ל-*true*.
- **TreeNum** - מטיפוס *int*. מתאר את מספר העצים שיש בערימה כלומר את מספר הצמתים שיש ברמה הגבוהה ביותר בערימה.
- **LinkNum** - שדה סטטי מטיפוס *int*. תפקידו לספור את מספר החיבורים שנעשו בכל הרצת התוכנית.
- **CutNum** - שדה סטטי מטיפוס *int*. תפקידו לספור את מספר החיתוכים שנעשו מתחילת התוכנית.

יתרה מכך, במחלקה זו מימשנו את המתודות הבאות:

○ **empty() -**

מה היא עושה-

מחזירה *true* אם הערימה ריקה אחרת יוחזר הערך *false*.

כיצד היא פועלת-

מחזירה את הערך *true* אם השדה *first* שווה ל-*null*.

סיבוכיות זמן הריצה שלה-

$O(1)$ שכן מתבצעת רק פעולה אחת לכל קלט.

○ size() -

מה היא עושה-

מחזירה את מספר האיברים שיש בערימה.

כיצד היא פועלת-

מחזירה את השדה size של הערימה.

סיבוכיות זמן הריצה-

$O(1)$ שכן מתבצע מספר קבוע של פעולות לכל קלט.

○ insert(int key) -

מה היא עושה-

מכניסה איבר בעל מפתח key לערימה.

כיצד היא פועלת-

ראשית ניצור צומת חדשה בעזרת הבנאי של HeapNode.

אם הערימה ריקה נעדכן את שדות ה-first וה-last להיות הצומת החדשה שנוצרה עם המפתח הנתון. אחרת, נכניס את הצומת לתחילת הרמה העליונה ואם המפתח של הצומת שהוכנסה קטן מהמינימום נעדכן את המינימום.

סיבוכיות זמן הריצה-

סיבוכיות זמן הריצה של הפונקציה insert במקרה הגרוע היא $O(1)$. זאת משום שבפעולת ההכנסה בוצע מספר סופי של פעולות במקרה הגרוע.

קריאה לפונקציות עזר-

כפי שכבר ציינו נעשתה קריאה לבנאי של HeapNode.

○ meld(FibonacciHeap heap2) -

מה היא עושה-

מחברת בין ערימת הפיבונאצי הנוכחית (this) לבין ערימת הפיבונאצי heap2.

כיצד היא פועלת-

אם אחת הערימות ריקות, יש לאתחל את שדות ה-first וה-last להיות השדות המתאימים של הערימה שאינה ריקה (אם שתיהן ריקות אין לעשות דבר). ואחרת, נעדכן את שדה ה-last ואת מצביע הערימה שלנו כך שאיברי heap2 יוצבו לאחר איברי הערימה הנוכחית. כל העבודה הינה באורך קבוע כאשר עובדים אך ורק עם קצוות הפוינטרים.

סיבוכיות זמן הריצה-

תמיד מבצעים מספר קבוע של פעולות ולכן במקרה הגרוע תהיה $O(1)$.

○ countersRep() -

מה היא עושה-

מחזירה מערך שמציין כמה עצים יש מכל דרגה. כלומר האיבר ה- i במערך יציין כמה עצים יש בדרגה i .

כיצד היא פועלת-

ראשית, נוצר מערך בגודל של הדרגה המקסימלית שיכולה להיות (לוגריתם בבסיס 1.6 של מספר הצמתים בעץ). בשלב זה עוברים על כל העצים בערימה ומגדילים עבור כל עץ את האיבר במערך במיקום של הדרגה של אותו עץ ב-1. לבסוף יוחזר המערך.

סיבוכיות זמן הריצה-

אנו נעבור על כל העצים בערימה ולכן הסיבוכיות תהיה לינארית במספר העצים.

○ findMin() -

מה היא עושה-

מחזירה את הצומת בערימה בעלת המפתח המינימלי או null במקרה בו הערימה ריקה.

כיצד היא פועלת-

מזמנת את הפונקציה `empty()` שבודקת אם הערימה ריקה. אם הערימה אינה ריקה יוחזר null ואם הערימה לא ריקה נבדוק את ערכו של השדה `beforemin` אם ערכו null תוחזר הצומת שנמצאת בשדה `first` אחרת תוחזר הצומת שנמצאת בשדה `next` של הצומת שנמצאת בשדה `beforeMin`.

סיבוכיות זמן הריצה-

$O(1)$ שכן מתבצע מספר קבוע של פעולות בפונקציה.

○ deleteMin()

מה היא עושה-

מוחקת את האיבר בעל המפתח המינימלי מהערימה אם קיים כזה.

כיצד היא פועלת-

ראשית, נבדוק האם הערימה ריקה בעזרת הפונקציה `empty`. אם היא ריקה לא נעשה כלום (אין איבר מינימלי). אחרת, בעזרת הפונקציה `findMin` נדע מי הצומת המינימלי. אם הוא היה מסומן נוריד את ערכו של השדה `marked` ב-1. אם ערכו של השדה `beforeMin` הוא `null` נדע שהאיבר המינימלי הוא האיבר שנמצא בשדה `first` ונפעל בהתאם. אחרת נבצע פעולת `meld` שמחברת את הערימה שנמצאת בשדה `child` של המינימלי עם הערימה שכוללת את העצים ללא העץ שהשורש שלו היה האיבר המינימלי. בפעולת ה-`meld` נעדכן את השדות של הערימה הקיימת לפי השדות של הערימה שהייתה בשדה `child` של האיבר המינימלי. כעת אם הערימה שנוצרה לא ריקה נבצע את פעולת ה-`consolidate` בעזרת הפונקציית עזר `consolidate`. בפעולת ה-`consolidate` אנו נשתמש באובייקט `Map` שערכיה- `key` שלו יהיו הדרגות של העצים השונים בערימה וערכי ה-`value` יהיו רשימות של כל הצמתים שנמצאים בשורש של העצים עבור הדרגה שנמצאת ב-`key`. נאחסן כפי שצינתי את ה-`Map` וכעת נבצע את פעולת ה-`consolidate` בכך שנחבר כל שני עצים בדרגה מסוימת וכך ייווצר עץ חדש בדרגה הגדולה ב-1 מהדרגה שהייתה לכל אחד משני העצים. בסיום התהליך מספר האיברים ב-`value` של כל דרגה יהיה במקסימום אחד. נוציא מהמילון ערכי `key` שהרשימות שלהם ריקות. לבסוף נשתמש בפעולת עזר `MapToHeap` כדי להעביר את ה-`Map` לייצוג ערימה שלנו.

סיבוכיות זמן הריצה-

$O(n)$ במקרה הגרוע שכן הסיבוכיות של פעולת ה-`consolidate` הוא $O(n)$ במקרה הגרוע ומלבדה אנו מבצעים מספר קבוע של פעולות.

קריאה לפונקציות עזר-

אנו השתמשנו במספר רב של פונקציות עזר משום שרצינו שהקוד יהיה ברור וקריא. פונקציות העזר בהן השתמשנו מפורטות תחת הסעיף "כיצד היא פועלת".

○ potential() -

מה היא עושה-

מחזירה את תוצאת החישוב $-treeNum + 2*marked$.

כיצד היא פועלת-

עושה את החישוב באמצעות השדות שבמחלקות.

סיבוכיות זמן הריצה-

$O(1)$ שכן מתבצע מספר קבוע של פעולות לכל קלט.

○ decreaseKey(HeapNode x, int delta) -

מה היא עושה-

פונקציה זו מורידה את ערכו של המפתח של צומת שהתקבלה כארגומנט בערך שלם שגם הוא התקבל.

כיצד היא פועלת-

ראשית, מתבצעת הפחתה של ערך המפתח של הצומת שהתקבלה בערך השלם שהתקבל. כעת יכול להיות ש"כלל הערימה" הופר ועל כן, אם כלל הערימה הופר אז תתבצע מחיקה של הצומת מהעץ בו היא נמצאת והשמתה כעץ חדש בערימה. כעת ניתן "לסמן" את האבא של אותה צומת שנמחקה ואם הוא כבר היה מסומן יש למחוק גם אותו וכן הלאה. מחיקת הצומת תתבצע באמצעות הפונקציה cut ומחיקת האבות המסומנים תתבצע באמצעות הפונקציה cascadingCut. הפונקציה cut תקבל את האיבר שאנו רוצים למחוק ואת ההורה שלו תמחק את האיבר מההורה שלו ותשים את האיבר להיות עץ חדש בערימה. הפונקציה cut משתמשת לפעולת המחיקה עצמה בפונקציה removeChild זו שממש מבצעת את המחיקה, זאת על מנת שהקוד יהיה מודלרי וקריא. כעת יש לעדכן את השדות של הערימה בהתאם. הפונקציה cascadingCut תבדוק אם האבא של הצומת מסומן ואז תמחק אותו עד אשר האבא של הצומת הנמחק כבר לא יהיה מסומן. פונקציה זו תעשה שימוש בפונקציה cut.

סיבוכיות זמן הריצה-

במקרה הגרוע הסיבוכיות תהיה כגובה הצומת שנמחקה מקרה זה יתרחש כאשר כל הצמתים עד הגובה מסומנים. הגובה במקרה הגרוע הוא $O(\log(n))$ ולכן זו תהיה הסיבוכיות במקרה הגרוע של פעולה זו.

קריאה למתודות עזר-

ישנה קריאה למספר פונקציות עזר על מנת שהקוד יהיה מודלרי וקריא. הסברנו על מתודות אלה תחת הכותרת "כיצד היא פועלת".

○ delete(HeapNode x) -

מה היא עושה-

מוחקת את הצומת x מן הערימה.

כיצד היא פועלת-

אם הערימה ריקה, הפעולה אינה עושה דבר. אחרת, היא מורידה את מפתח הצומת x כך שהמפתח שלו יהיה המינימלי באמצעות קריאה לפונקציה decreaseKey. לאחר מכן, נעשית קריאה לפונקציה deleteMin אשר תמחק את x שכן כעת הוא האיבר המינימלי.

סיבוכיות זמן הריצה-

$O(n)$ במקרה הגרוע שכן הסיבוכיות של פעולת ה-deleteMin הוא $O(n)$ במקרה הגרוע ומלבדה אנו קוראים לפעולה decreaseKey אשר סיבוכיותה במקרה הגרוע הוא $O(\log(n))$ ולכן בסה"כ הסיבוכיות במקרה הגרוע הוא $O(n)$.

○ totalLinks() -

מה היא עושה-

מחזירה את מספר הלינקים שנעשו מתחילת הרצת התוכנית.

כיצד היא פועלת-

מחזירה את השדה linkNum של הערימה.

סיבוכיות זמן הריצה-

$O(1)$ שכן מתבצע מספר קבוע של פעולות לכל קלט.

○ totalCut() -

מה היא עושה-

מחזירה את מספר החיתוכים שנעשו מתחילת הרצת התוכנית.

כיצד היא פועלת-

מחזירה את השדה cutNum של הערימה.

סיבוכיות זמן הריצה-

$O(1)$ שכן מתבצע מספר קבוע של פעולות לכל קלט.

מדידות

Sequence 1:

הרץ את סדרת הפעולות הבאות (הרצה חדשה עבור $m=1000$, $m=2000$, $m=3000$)

Insert(m), insert($m-1$), ..., insert(1)

Potential	totalLinks	totalCuts	זמן ריצה	ערך M	הרצה מספר
1000	0	0	1	1000	1
2000	0	0	1	2000	2
3000	0	0	2	3000	3

1. זמן הריצה האסימפטוטי הוא $O(m)$ הסיבה לכך היא אמנם שכל פעולת Insert מתבצעת בסיבוכיות $O(1)$ כפי שכבר ציינו בתיעוד פונקציה זו.

2. מהסתכלות בפקודות התכנית ניתן להיווכח כי לא מתבצעות כלל פעולות link ו cut.

3. ראשית, אשים לב כי מכיוון שיש רק הכנסות מתקיים אמנם שמספר העצים כמוהו מספר הצמתים שהוכנסו. מסיבה זו, הפוטנציאל יהיה $O(m)$ שכן הפוטנציאל מייצג את מספר העצים והוכנסו m איברים שכל אחד מהם יהיה עץ בערימה.

Sequence 2:

הריץ את סדרת הפעולות הבאות (הריצה חדשה עבור $m=1000$, $m=2000$, $m=3000$)

Insert(m), insert($m-1$), ..., insert(1)

deleteMin(), deleteMin (), ..., deleteMin() (run delete min $m/2$ times).

Potential	totalLinks	totalCuts	זמן ריצה	ערך M	הריצה מספר
6	1891	0	18	1000	1
6	5780	0	12	2000	2
7	11552	0	8	3000	3

1. זמן הריצה האסימפטוטי של סדרת הפעולות המצוינת לעיל הוא $O(m \log m)$. הסיבה היא שכל פעולת הכנסה מתבצעת ב $O(1)$ כפי שכבר פירטנו לעיל תחת הכותרת insert. פעולת מחיקת המינימום מתבצעת בזמן אמורטייז של $O(\log m)$ כפי שלמדנו בהרצאה ולכן אוכל להכפיל את מספר הפעולות באמורטייז של כל פעולה ולקבל את התוצאה שקיבלתי.

2. לא מתבצעות אף פעולות cut שכן רק פעולות decrease key או delete מבצעות cut כפי שציינתי כבר לעיל בתיעוד הקוד.

אין בחלק זה פעולות decrease key ולכן יש התנהגות של ערימה בינומית עצלה!

ניתוח מספר פעולות ה-link

בחלק זה אין פעולות decrease key ולכן ערימת פיבונצ'י מתנהגת כמו ערימה בינומית עצלה. הכנסנו לערימה זו את האיברים 1 עד m בסדר יורד. משום כך אחרי פעולת ה deleteMin הראשונה שבאה למחוק איבר שהמפתח שלו הוא 1. נקבל שעבור שני עצים מדרגות שונות הערכים בעץ שדרגתו נמוכה קטנים מהערכים בעץ השני. כמו כן, ניתן להיווכח כי סיבוכיות פעולת הדיליט מין הראשונה היא $O(m)$ וניתן לראות זאת באמצעות חישוב מספר אמורטייז של החיבורים.

לצורך הוכחה זו נגדיר פונקצית פוטנציאל באופן הבא-

הפוטנציאל שווה למספר העצים פחות מספר העצים מדרגה 1.

עבור כל צומת שנחבר עם צומת אחרת ירידת מספר העצים תצטמצם עם מספר החיבורים שביצענו עד כה למעט הראשון ולכן נקבל אמורטייז של $O(1)$ חיבורים.

ולכן עבור סדרה של M חיבורים סיבוכיות זמן הריצה תהיה $O(m)$.

לאחר מכן כל פעולת deleteMin נוספת לא תבצע אף חיבור מכיוון שכאשר ביצענו מחיקה של המינימום פירקנו את העץ מהדרגה הקטנה ביותר משום שהמינימום שם ולכן קיבלנו עצים כך שכל עץ מדרגה קטנה מדרגת העץ לפני אותו פירוק.

יתרה מכך, נשים לב שכתוצאה מהדברים המובאים לעיל יתקיים כי לא קיימים עצים מאותה דרגה. מכך שעשינו חיבור שניתן לעשות לו רדוקציה שחיבור בינארי, מאופן החיבור, המינימום החד ששל הערימה יהיה שורש העץ עם הדרגה המינימלית והתהליך ימשיך ככה ללא חיבורים נוספים. כלומר, בסך הכל, ניתן להסתכל על הפועולות שביצענו כמחיקה ופיצול של העצים מהדרגה הנמוכה לפני שמטפלים בדרגות הגבוהות יותר.

נשים לב כי הניתוח שעשינו תואם את התוצאות שקיבלנו במדידותינו אשר הובאו אמנם בטבלה הערוכה לעיל.

3. אבצע כעת ניתוח של הפוטנציאל

אשים לב כי בשלב זה ניתן להתייחס אל הערימה כבינומית כי כבר דיברתי לעיל על כך שהיא עצלה וכי לא קיימים שני צמתים מאותה דרגה.

הפוטנציאל מייצג אמנם את מספר העצים בערימה. אחרי פעולת המחיקה נישאר עם $m/2$ איברים. מספר העצים הדרוש למספר צמתים זה הוא כמספר האחדים בייצוג הבינארי של $m/2$. מספר זה חסום על ידי $O(\log m)$.

כעת אעבור על הערכים השונים של m :

$M=1000$

אחרי מחיקת 500 צמתים נישאר עם 500 צמתים. הייצוג הבינארי של 500 הוא 111110100 ישנם שישה אחדים בייצוג זה ולכן הפוטנציאל הוא 6 שכן יש 6 עצים בערימה.

$M=2000$

אחרי מחיקת 1000 צמתים נישאר עם 1000 צמתים. הייצוג הבינארי של 1000 הוא 1111101000 יש בייצוג זה 6 אחדות ולכן הפוטנציאל הוא 6 שכן יש 6 עצים בערימה.

$M=3000$

אחרי מחיקת 1500 צמתים נישאר עם 1500 צמתים. הייצוג הבינארי של 1500 הוא 10111011100 יש בייצוג זה 7 אחדות ולכן הפוטנציאל הוא 7 שכן יש 7 עצים בערימה.